CSE 4303
Data Structures

Topic: Trie

Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu

- Using a *Trie* (AKA *prefix tree*), you can easily find strings in a database.

- Using a *Trie* (AKA *prefix tree*), you can easily find strings in a database.

- Suppose a phone book contains the phone numbers of all the people in a city. There may be hundreds of thousands of people in the city.
- Let, each person's name can be written using a maximum of 20 characters.

- Using a *Trie* (AKA *prefix tree*), you can easily find strings in a database.

- Suppose a phone book contains the phone numbers of all the people in a city. There may be hundreds of thousands of people in the city.
- Let, each person's name can be written using a maximum of 20 characters.

- Trie can *search for names based solely on the number of characters in the name*.
- For example, searching for the name ‘Alice’ will require only 5 operations, no matter how large the database is.

- Search: *O(length of string)*

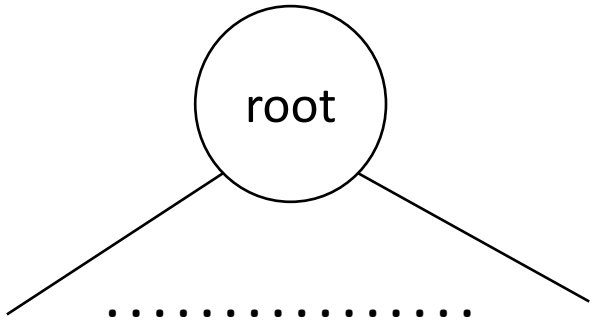
- Tries are an extremely special and useful data-structure that are based on the *prefix of a string*.
- Represents the “Re ***trie*** val” of data and thus the name Trie.

- Tries are an extremely special and useful data-structure that are based on the *prefix of a string*.
- Represents the “Re ***trie*** val” of data and thus the name Trie.

- **Prefix of a string:** substrings that strictly begins from the first letter

- For example , the word “abacaba” has the following prefixes:
  a
  ab
  aba
  abac
  abaca
  abacab

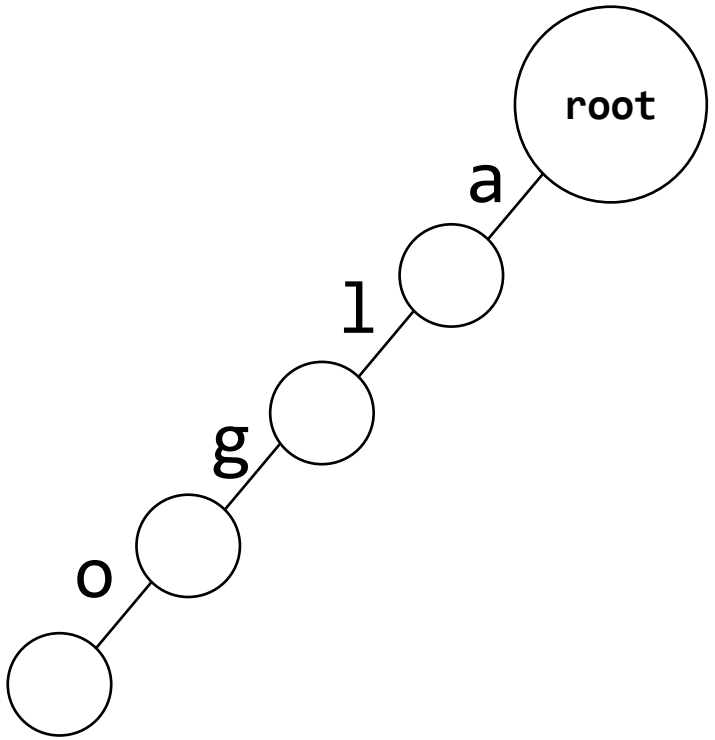
- Trie stores strings that can be visualized like a graph.
- Consists of nodes and edges.
- *Each node consists of at max 26 children* and edges connect each parent node to its children. (*26 is the Alphabet size, may vary*)
- These 26 pointers are nothing but pointers for each of the 26 letters of the English alphabet A separate edge is maintained for every edge.



Pointers for 26 possible children

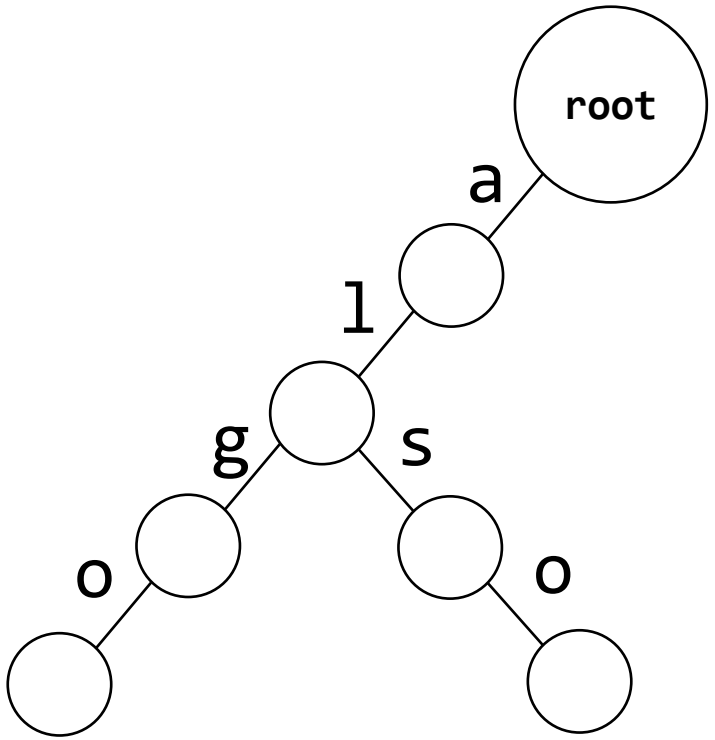
Next[0] is reserved for ‘A’
...
Next[25] is for ‘z’

If alphabet size is bigger, need more pointers



Insert ‘algo’

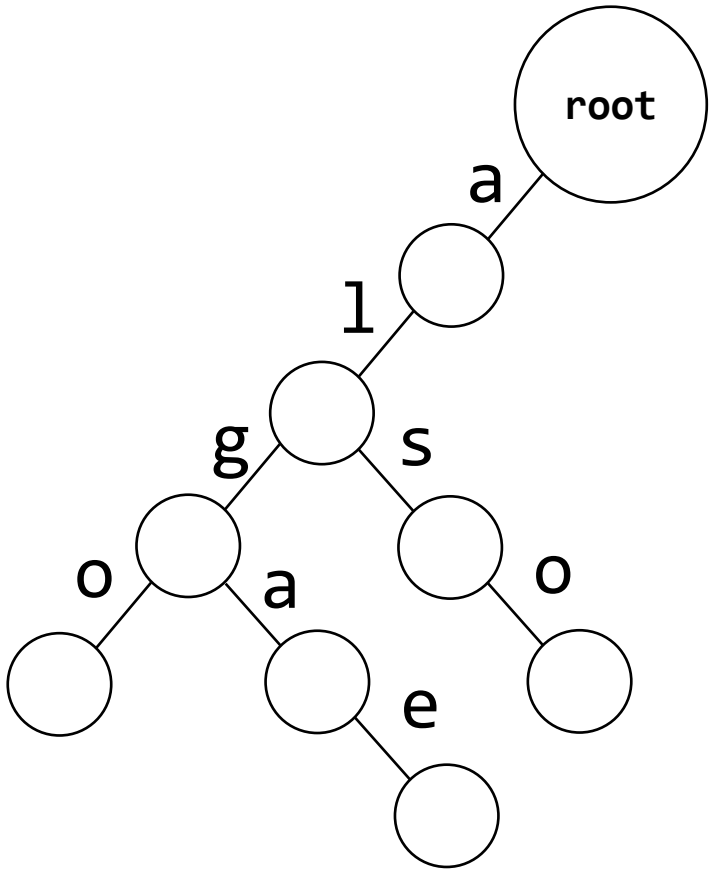
From root, create pointer for ‘a’
From ‘a’ create pointer for ‘l’
. . .



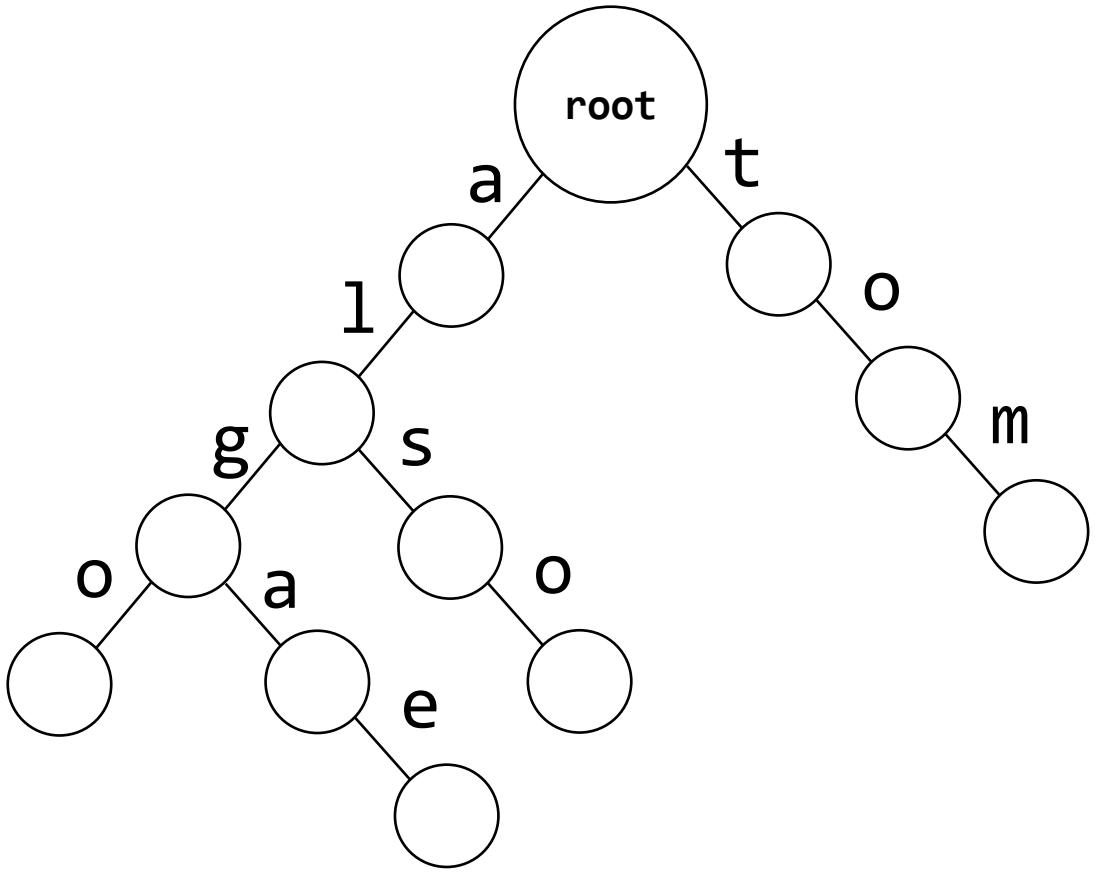
Insert ‘also’

From root, already pointer for ‘a’
Goto a
From ‘a’, already pointer for ‘l’
Goto ‘l’

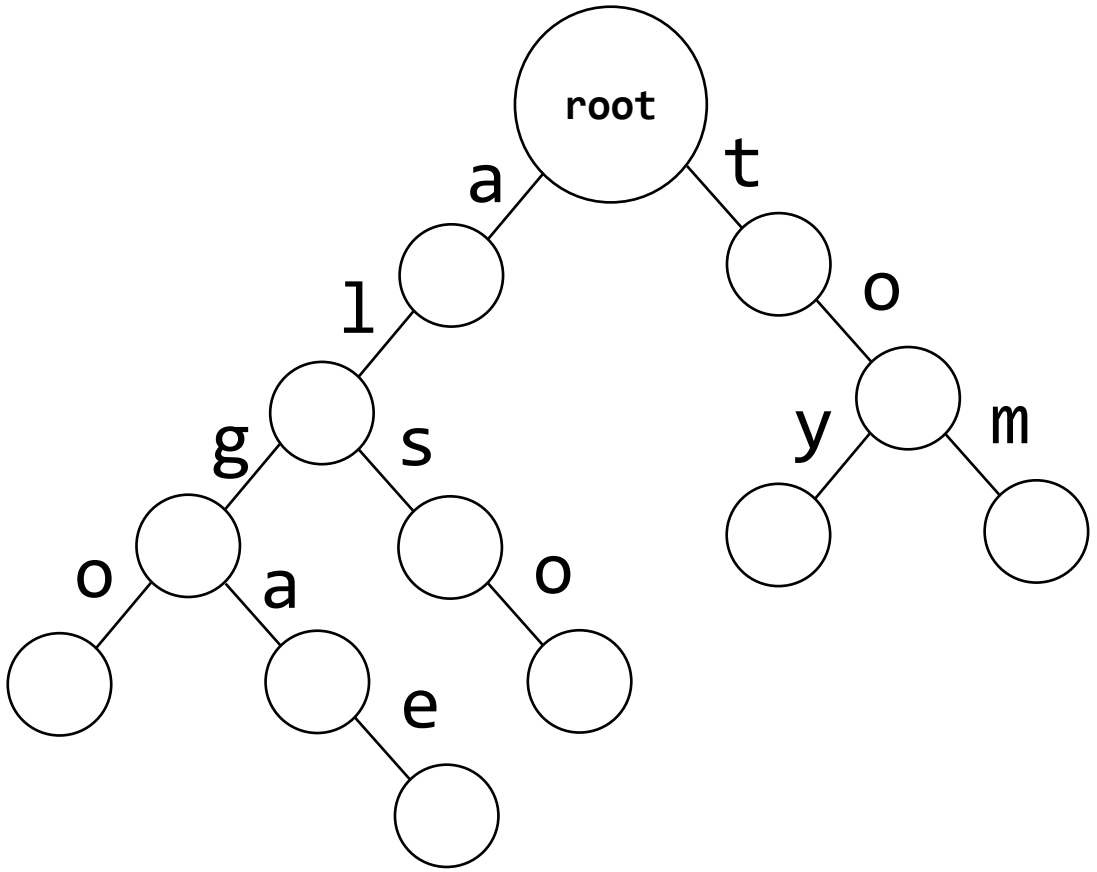
From ‘l’ create pointer for ‘s’
From ‘s’ create pointer for ‘o’



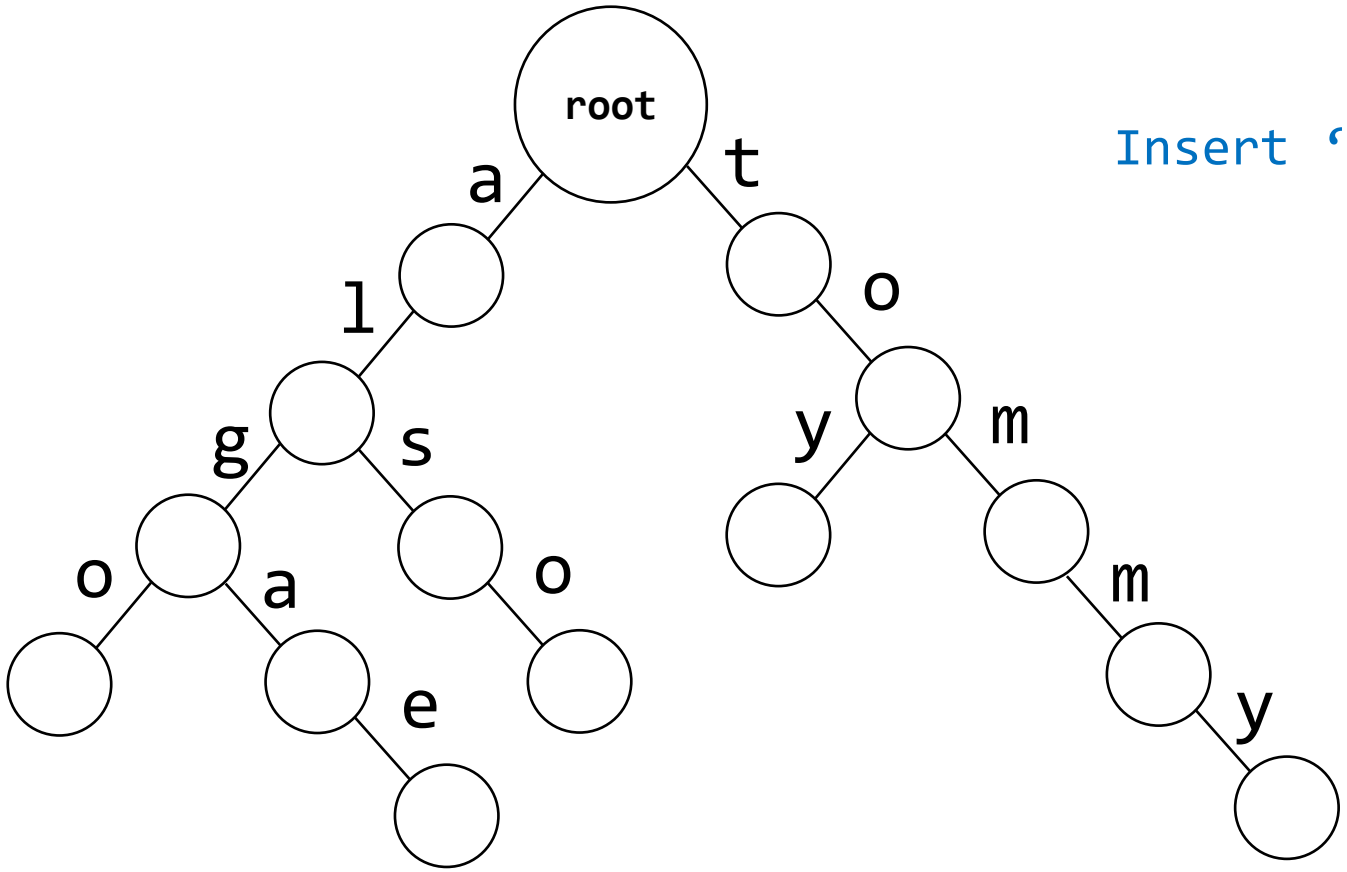
Insert ‘algae’



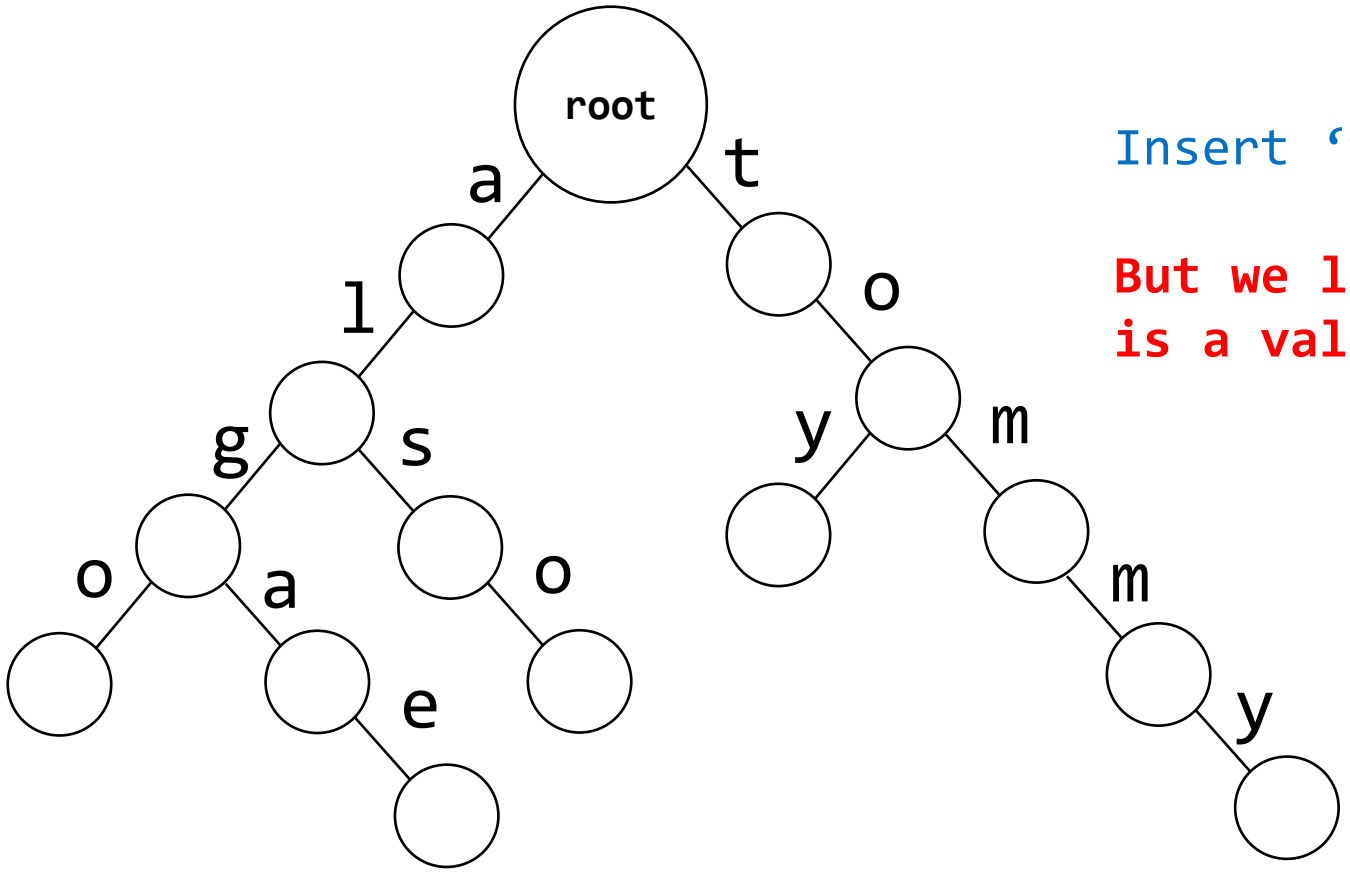
Insert ‘tom’



Insert ‘toy’

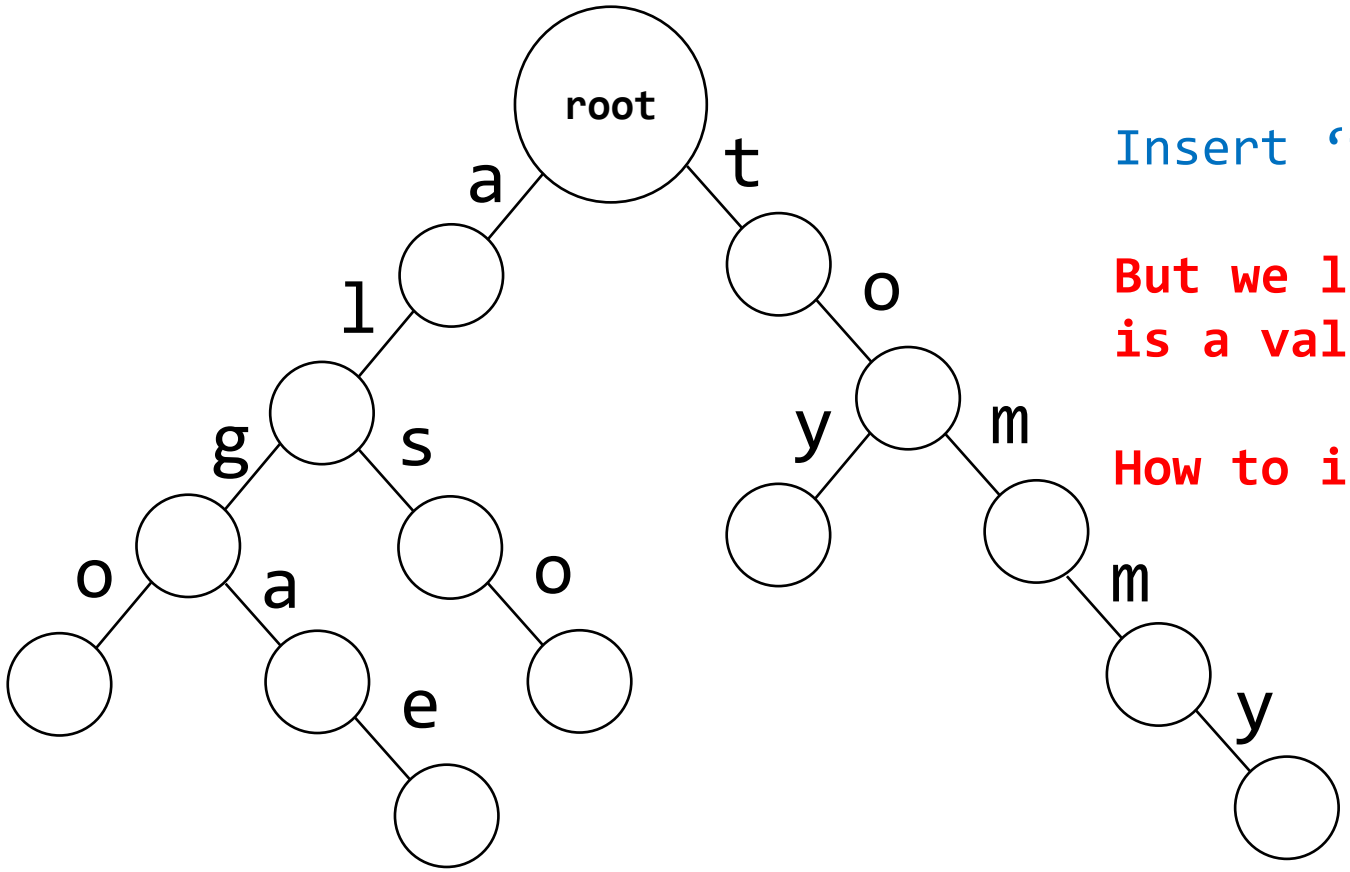


Insert ‘tommy’



Insert ‘tommy’

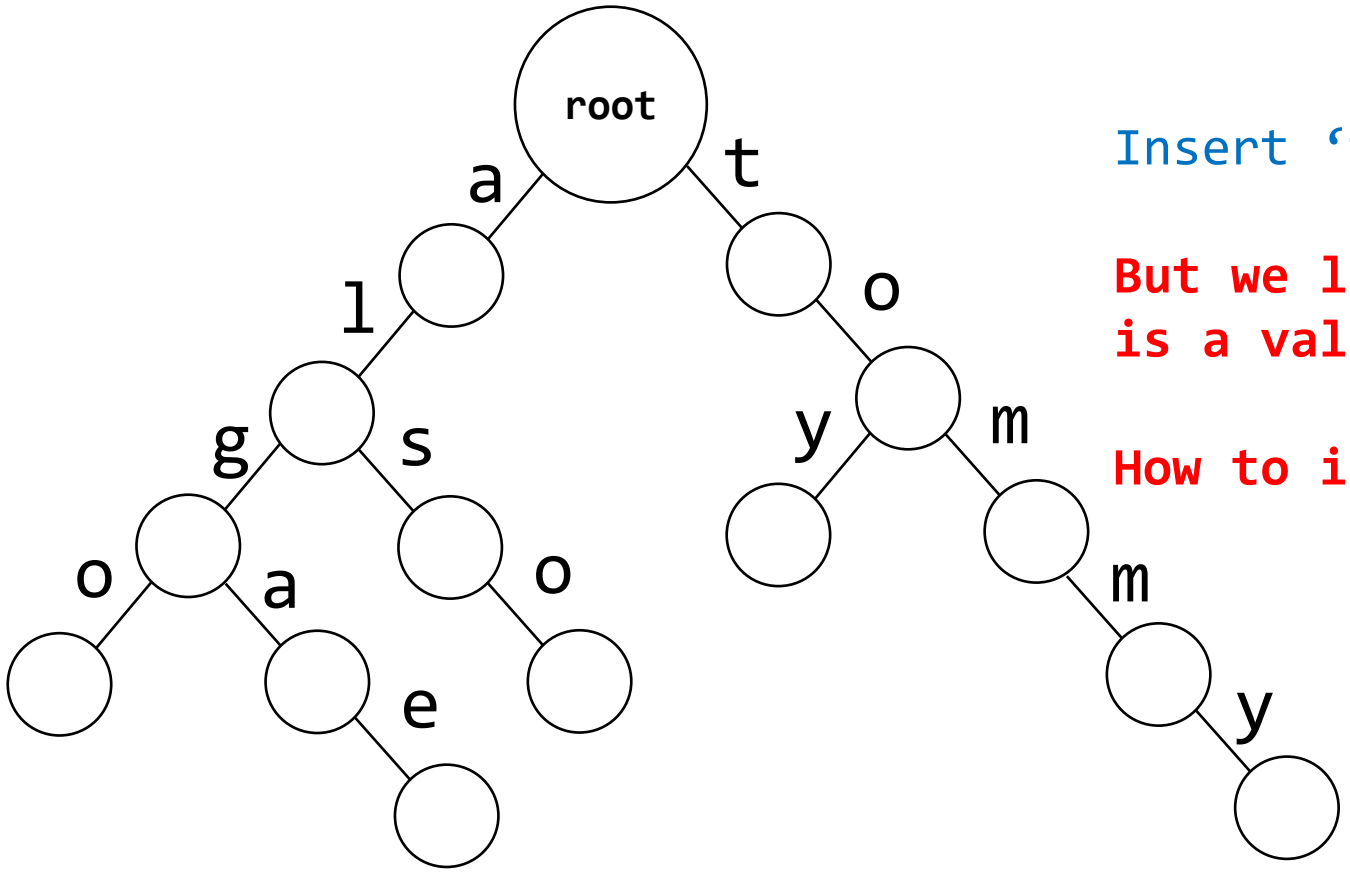
But we lost information that ‘tom’ is a valid word



Insert ‘tommy’

But we lost information that ‘tom’ is a valid word

How to insert ‘to’



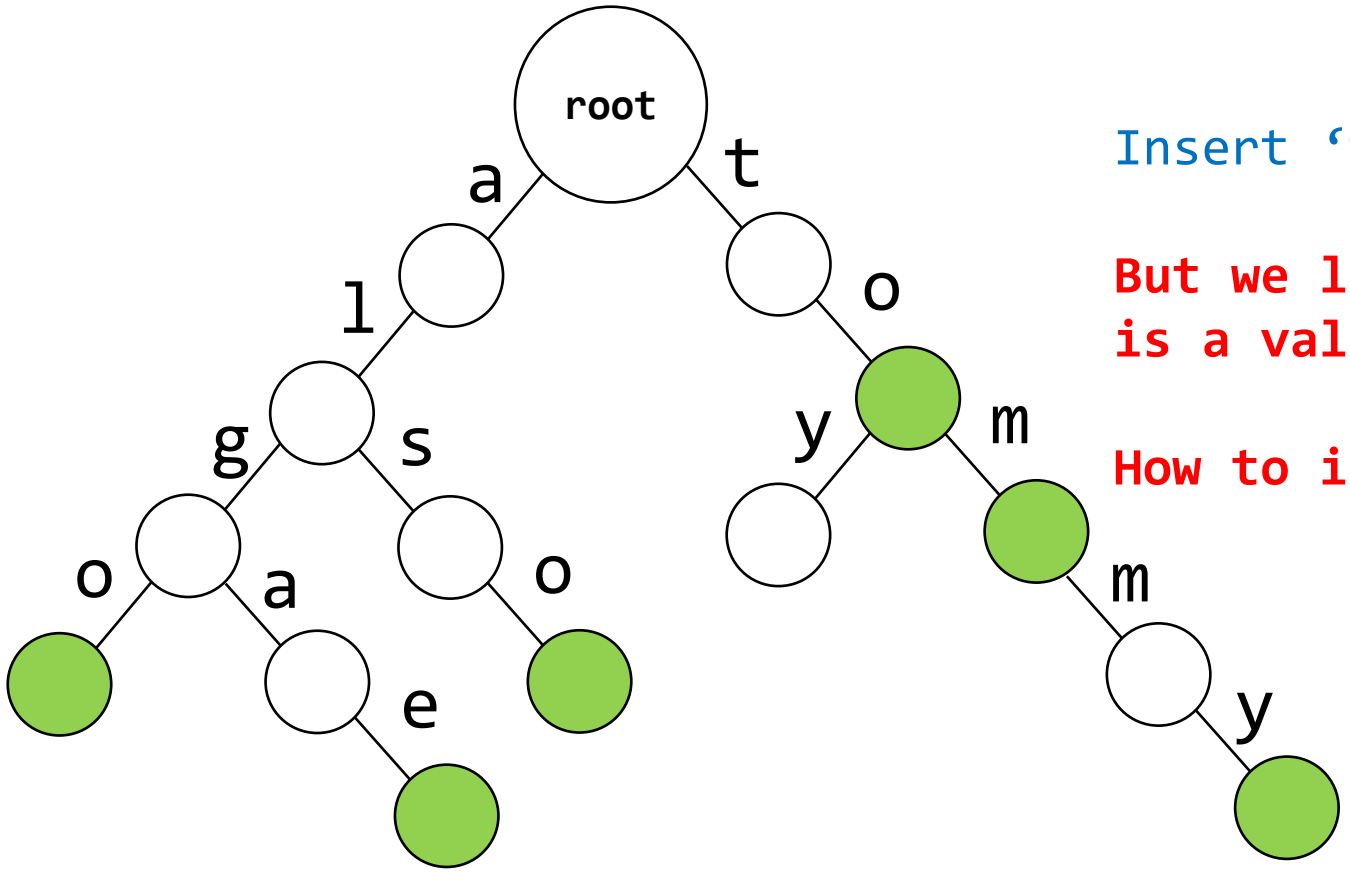
Insert ‘tommy’

But we lost information that ‘tom’ is a valid word

How to insert ‘to’

Idea:

Node can have an **endmark** where a word stops!



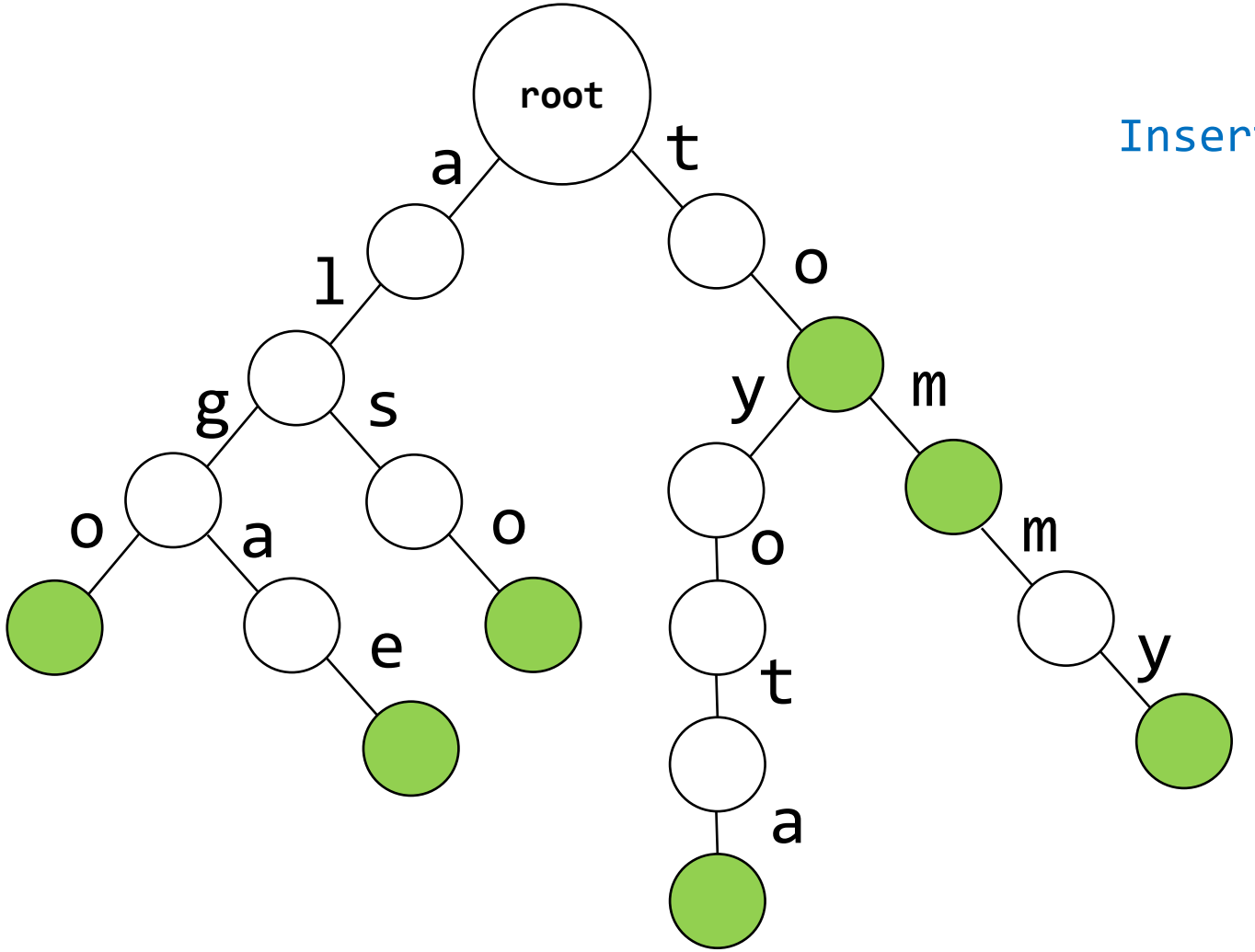
Insert ‘tommy’

But we lost information that ‘tom’ is a valid word

How to insert ‘to’

Idea:

Node can have an **endmark** where a word stops!



Insert ‘Toyota’

```
struct node {
    bool endmark;
    node* next[26];

}
```

```
struct node {
    bool endmark;
    node* next[26];
    node() {
        endmark = false;
        for (int i = 0; i < 26; i++)
            next[i] = NULL;
    }
} * root;
```

```
struct node {
    bool endmark;
    node* next[26];
    node() {
        endmark = false;
        for (int i = 0; i < 26; i++)
            next[i] = NULL;
    }
} * root;

int main() {
    root = new node();
    return 0;
}
```

```
void insert(char* str, int len)
{
    node* curr = root;
    for (int i = 0; i < len; i++) {
        int idx = str[i] - 'a';









}
```

```
void insert(char* str, int len)
{
    node* curr = root;
    for (int i = 0; i < len; i++) {
        int idx = str[i] - 'a';

        if (curr->next[idx] == NULL)
                                ?



}
```

```
void insert(char* str, int len)
{
    node* curr = root;
    for (int i = 0; i < len; i++) {
        int idx = str[i] - 'a';

        if (curr->next[idx] == NULL)
            curr->next[idx] = new node();




}
```

```
void insert(char* str, int len)
{
    node* curr = root;
    for (int i = 0; i < len; i++) {
        int idx = str[i] - 'a';

        if (curr->next[idx] == NULL)
            curr->next[idx] = new node();

        curr = curr->next[idx];
    }
}
```

Anything else?

```
void insert(char* str, int len)
{
	node* curr = root;
	for (int i = 0; i < len; i++) {
		int idx = str[i] - 'a';

		if (curr->next[idx] == NULL)
			curr->next[idx] = new node();

		curr = curr->next[idx];
	}
	curr->endmark = 1;
}
```



```
bool search(char* str, int len)
{
    node* curr = root;
    for (int i = 0; i < len; i++) {
        int id = str[i] - 'a’;

        if (curr->next[id] == NULL)
                        ??




}
```

```
bool search(char* str, int len)
{
    node* curr = root;
    for (int i = 0; i < len; i++) {
        int id = str[i] - 'a’;

        if (curr->next[id] == NULL)
            return false;




}
```

```
bool search(char* str, int len)
{
    node* curr = root;
    for (int i = 0; i < len; i++) {
        int id = str[i] - 'a’;

        if (curr->next[id] == NULL)
            return false;

        curr = curr->next[id];
    }
```

??

```
}
```

```
bool search(char* str, int len)
{
    node* curr = root;
    for (int i = 0; i < len; i++) {
        int id = str[i] - 'a’;

        if (curr->next[id] == NULL)
            return false;

        curr = curr->next[id];
    }
    return curr->endmark;
}
```

- How to display all the words of a trie?

- Applications:
  - How many words start with a certain prefix?
  - Print the words starting with a given prefix
  - Phonebook autocomplete feature
  - Given a set of numbers, find the Maximum XOR between of a pair

References

1. GeeksforGeeks. (2025, February 12). Trie Data Structure | Insert and search. https://www.geeksforgeeks.org/trie-insert-and-search/
2. https://www.hackerearth.com/practice/data-structures/advanced-data-structures/trie-keyword-tree/tutorial/
3. Ashraf, S. (2020, June 7). ডাটা স্ট্রাকচার: ট্রাই (প্রিফিক্স ট্রি/রেডিক্স ট্রি). শাফায়েতের ব্লগ. https://www.shafaetsplanet.com/?p=1679

Thank you